ARTIFICIAL INTELLIGENCE

TOPIC 5

Advanced deep learning

Explanatory study notes

Theory, worked examples and revision questions

What you should be able to do

Understand sequence models and LSTM gates; compute small attention examples and parameter counts; compare transformer families and foundation models; explain unsupervised and self-supervised representations; distinguish autoencoders, VAEs, GANs and diffusion. This volume covers all of slides 5.1–5.5, including the LSTM extra material.

Based on the teacher PDFs supplied in `Lecture Slides`. The notes explain and reorganize that material. Worked calculations are study examples derived from the slide concepts, unless explicitly identified as a slide exercise. Clarifications of ambiguous or incorrect expressions are marked in context. References use **PDF page numbers**, counted from 1, rather than printed slide numbers.

English edition • 24 October 2026

Contents

5.1 Sequence models and recurrent networks 4
5.1.1 Sequential data: order and context 4
5.1.2 A vanilla RNN: equations, states and predictions 4
5.1.3 Training through time 5
5.1.4 LSTM: a memory state controlled by gates 6
5.1.5 LSTM sizes and parameters: the extra-material exercise 7
5.1.6 Worked slide exercise: reading the letter b 7
5.1.7 GRUs, stacked networks and bidirectional processing 8
5.1.8 Temporal convolutions 8
5.1.9 Encoder–decoder models and teacher forcing 9
5.1.10 Revision questions and answers 10
5.1.11 Source guide 11
5.2 Attention and transformers 12
5.2.1 Why attention? 12
5.2.2 Queries, keys and values 12
5.2.3 Small attention calculations 13
5.2.4 Order: positional information 14
5.2.5 Tokenization, embeddings and context 15
5.2.6 Inside a transformer block 15
5.2.7 Three transformer families 16
5.2.8 Computation and memory 17
5.2.9 Vision transformers 18
5.2.10 Worked classroom exercise: A or B? 18
5.2.11 Revision questions and answers 19
5.2.12 Source guide and connection to Lab 5 19
5.3 Foundation models 21
5.3.1 What makes a model a foundation model? 21
5.3.2 Architecture, objective and adaptation 21
5.3.3 Where the training signal comes from 22
5.3.4 Scale, storage and mixture of experts 23
5.3.5 CLIP: align images and text 24
5.3.6 DINOv2 and SAM 24
5.3.7 Language-conditioned image generation 25

5.3.8 Benefits and limitations to explain . . . 25
5.3.9 Revision questions and answers . . . 26
5.3.10 Source guide . . . 27
5.4 Unsupervised and self-supervised learning . . . 28
5.4.1 Learning without task labels . . . 28
5.4.2 PCA: compress along directions of variation . . . 28
5.4.3 t-SNE: visualize local neighborhoods . . . 29
5.4.4 K-means: assignments and means . . . 30
5.4.5 Hierarchical clustering and topic mixtures . . . 31
5.4.6 Self-supervision through proxy tasks . . . 32
5.4.7 Contrastive representations . . . 32
5.4.8 BYOL, SuperPoint and semi-supervised extensions . . . 33
5.4.9 Revision questions and answers . . . 34
5.4.10 Source guide . . . 35
5.5 Generative models . . . 36
5.5.1 Discriminative and generative modeling . . . 36
5.5.2 Autoencoders: learn to reconstruct . . . 36
5.5.3 Variational autoencoders . . . 37
5.5.4 GANs: a learned adversarial signal . . . 38
5.5.5 GAN difficulties and image translation . . . 39
5.5.6 Diffusion: learn to reverse a noise process . . . 39
5.5.7 Conditioning and latent diffusion . . . 40
5.5.8 Compare the model families . . . 41
5.5.9 Place these models in the course . . . 41
5.5.10 Revision questions and answers . . . 41
5.5.11 Source guide . . . 43

5.1 Sequence models and recurrent networks

5.1.1 Sequential data: order and context

Teacher slides: 5.1. Sequence Models, pp. 3–6.

A sequence is an ordered collection $(x_1, \ldots, x_T)$. Reordering words can change a sentence's meaning; reversing video frames can change the action. A useful model must account for these relationships rather than treating each item as an unrelated sample.

Different tasks have different input/output arrangements. Sentiment classification maps many words to one label. Video labeling may map one sequence to a label at every time step. Translation maps an input sentence to an output sentence of a possibly different length. Image captioning maps an image representation to a sequence of words.

A recurrent neural network (RNN) processes one element at a time while maintaining a hidden state. The same transition is reused for each step, so the model can process different sequence lengths without introducing a new set of weights for each position.

Variable length is not unique to RNNs

Dense layers require a specified feature dimension, but convolutional architectures can also accept variable spatial or temporal sizes when their readout permits it. The important RNN distinction here is its recurrent state and shared transition, not that every other architecture must have a fixed number of inputs.

5.1.2 A vanilla RNN: equations, states and predictions

Teacher slides: 5.1. Sequence Models, pp. 6–12.

Using column vectors, input size D, hidden size H, and output size O:

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h), \qquad s_t = W_{hy}h_t + b_y.$$

The hidden state $h_t \in \mathbb{R}^H$ summarizes the current input and previous state. For classification or next-token prediction, the scores s_t usually pass through softmax to obtain probabilities. Scores themselves need not lie in $[0, 1]$ or sum to one.

Object	Shape	Role
x_t	$D \times 1$	Current observation or token representation.
W_{xh}	$H \times D$	Maps input features into the state update.
W_{hh}	$H \times H$	Maps the previous hidden state into the update.
b_h	$H \times 1$	Hidden bias.
h_t	$H \times 1$	Computed state, different at different steps.
W_{hy}	$O \times H$	Maps the state to output scores.
b_y	$O \times 1$	Output bias.

Parameters are learned; states are computed

During an ordinary forward pass with fixed model weights, x_t, h_t, scores and predictions can change at every step. The shared weight matrices do not change merely because the next token arrives. An optimizer updates parameters during training, after gradients have been

computed. A hidden-state entry is a learned feature, not automatically a particular word or class.

Character-level example

For a vocabulary of V characters, a one-hot input has $D = V$ and a next-character output has $O = V$. Multiplying W_{xh} by a one-hot vector selects one column of W_{xh}. That column is the input contribution associated with the character; the final state also depends on the preceding context through $W_{hh}h_{t-1}$.

The slide examples show units responding to patterns such as URLs or bracket structure. This illustrates that learned states can track useful context. It does not imply that every unit has a simple human-readable meaning or that the network contains a separate unit for every possible concept.

Parameter count

With one hidden bias and one output bias,

$$N_{\text{RNN}} = HD + H^2 + H + OH + O.$$

For $V = 3, H = 2$, a next-character model has $6 + 4 + 2 + 6 + 3 = 21$ parameters. Processing 10 rather than 100 characters changes the number of operations and stored training activations, not this count. The slide's displayed vanilla-RNN equations omit biases; if a question omits them explicitly, remove the corresponding terms.

5.1.3 Training through time

Teacher slides: 5.1. Sequence Models, pp. 13–16, 32.

Unrolling and shared gradients

Unroll the recurrence over T steps. This gives a computation graph with T copies of the transition operation, all using the same parameters. A common objective is

$$L = \sum_{t=1}^{T} \ell_t, \qquad \ell_t = -\log p_\theta(y_t^* \mid x_{\le t}).$$

Backpropagation through time (BPTT) applies the chain rule to that unrolled graph. Because the same weight contributes at many time steps, its gradient accumulates contributions from those uses. It is still one trainable parameter.

Why gradients vanish or explode

A distant state's influence contains a product of Jacobians:

$$\frac{\partial h_t}{\partial h_k} = J_t J_{t-1} \cdots J_{k+1}, \qquad J_t = \operatorname{diag}(1 - h_t^2)\, W_{hh}.$$

Repeated multiplication by factors with small norms can make the gradient tiny. Factors with large amplification can cause it to explode. Saturated tanh units also have small derivatives. For a scalar illustration, ten factors of 0.5 yield $0.5^{10} \approx 0.00098$; ten factors of 2 yield 1024.

The slides' singular-value argument is a bound: a largest singular value below one gives a contraction bound for this vanilla transition. A value above one permits growth; it does not guarantee that every gradient grows, because activation derivatives and directions also matter.

Clipping, truncation and architectural changes

Norm clipping rescales a gradient vector g when $\|g\| > \tau$:

$$g_{\text{clip}} = g \min\left(1, \frac{\tau}{\|g\|}\right).$$

For $g = (3, 4)$ and threshold $\tau = 2$, the norm is 5 and the clipped vector is $(1.2, 1.6)$. Its direction is unchanged. This limits an exploding gradient; it does not recover a gradient that has vanished. The parameter step also depends on the optimizer.

Truncated BPTT limits differentiation to a window of K steps. A state may be carried forward while its earlier computation history is detached. This saves memory and computation, but cuts the gradients needed to assign credit farther back. LSTM/GRU gates, suitable initialization and normalization address different aspects of recurrent optimization; they are not interchangeable with truncation.

5.1.4 LSTM: a memory state controlled by gates

Teacher slides: 5.1. Sequence Models, pp. 17–21; 5.1. Extra Material, pp. 2–4.

An LSTM has both a cell state c_t and a hidden state h_t, each of dimension H. For each gate/candidate $r \in \{f, i, g, o\}$, form

$$a_r = W_{xr} x_t + W_{hr} h_{t-1} + b_r.$$

Then

$$\begin{aligned} f_t &= \sigma(a_f), & i_t &= \sigma(a_i), \\ g_t &= \tanh(a_g), & o_t &= \sigma(a_o), \\ c_t &= f_t \odot c_{t-1} + i_t \odot g_t, & h_t &= o_t \odot \tanh(c_t). \end{aligned}$$

Here $\odot$ is elementwise multiplication. The sigmoid gates lie between 0 and 1; the candidate g_t lies between -1 and 1. The cell state is not restricted to either interval.

Quantity	Interpretation
Forget gate f_t	Fraction of the old cell content retained. Near 1 keeps it; near 0 removes it.
Input gate i_t	How much of the new candidate is written.
Candidate g_t	Proposed signed content, not itself a probability.
Output gate o_t	How much of the transformed cell state is exposed as h_t.

The additive cell update supplies a path that can preserve information. Along the direct cell-to-cell path, the derivative is the forget gate: if it stays near one, that path avoids repeated multiplication by a dense recurrent matrix. Gates help with long dependencies, but do not guarantee unlimited memory or perfect gradients.

One small update

Suppose one unit has $c_{t-1} = 2$, $f_t = 0.5$, $i_t = 0.25$, $g_t = 0.8$, and $o_t = 1$. Then

$$c_t = 0.5(2) + 0.25(0.8) = 1.2, \qquad h_t = \tanh(1.2).$$

The old contribution is 1 and the newly written contribution is 0.2. Closing the output gate would hide the cell from the output without necessarily erasing the stored cell value.

5.1.5 LSTM sizes and parameters: the extra-material exercise

Teacher slides: 5.1. Extra Material, pp. 5–6.

The class exercise uses a one-hot vocabulary $\{a, b, c\}$, so $V = 3$, with $H = 2$. Adopt the column-vector convention above and one combined bias vector per gate.

Object	Shape	Meaning
x_t	3×1	Selects the current letter.
W_{xf}, W_{hf}, b_f	2×3, 2×2, 2	Forget-gate input weights, recurrent weights and bias.
Stacked W_x	8×3	Four blocks, each with two units.
Stacked W_h	8×2	Four recurrent blocks.
Stacked bias	8	Two biases in each of four blocks.
c_t, h_t	2×1 each	Computed memory and exposed state.
W_y, b_y	3×2, 3	Next-character output head.

The LSTM cell has

$$\boxed{N_{\text{LSTM}} = 4(HD + H^2 + H) = 4H(D + H + 1).}$$

For $D = 3, H = 2$, this is $4(6 + 4 + 2) = 48$. The output head adds $3(2 + 1) = 9$, giving 57 in total. Implementations with separate input and recurrent bias vectors count extra stored bias parameters; a counting question must state its convention.

The input-specific contribution of letter b is stored in column b of stacked W_x: eight numbers here. Its contextual effect also depends on recurrent weights and the current state. Its output score has its own row in W_y and output bias; input encoding and output prediction are different roles.

Adding a fourth letter while keeping $H = 2$ expands x_t, W_x and the output head. It does not change the lengths of c_t, h_t or the shape of W_h. The cell now has $4(8 + 4 + 2) = 56$ parameters. Conversely, keeping $V = 3$ and doubling H to 4 gives $4(12 + 16 + 4) = 128$ cell parameters, plus 15 output-head parameters. Doubling hidden width is more than a doubling of the recurrent parameter count because of the H^2 term.

5.1.6 Worked slide exercise: reading the letter b

Teacher slides: 5.1. Extra Material, p. 7.

This is the teacher's numerical exercise, using the supplied rounded sigmoid and tanh values. Start with $c_{t-1} = (1, 0.5)$, $h_{t-1} = (0.5, 0)$ and $x_t = (0, 1, 0)$. The one-hot input selects the second column of each input matrix.

Applying the displayed weights gives:

$$\begin{aligned} a_f &= (2 + 4(0.5),\ -4 + 2(0)) = (4, -4), \\ a_i &= (-4 + 0,\ 4 + 0) = (-4, 4), \\ a_g &= (0,\ 2 - 2(0.5)) = (0, 1), \\ a_o &= (0 + 3(0),\ 0 + 2(0.5) - 1) = (0, 0). \end{aligned}$$

The gates and candidate are therefore approximately

$$f = (0.98, 0.02), \quad i = (0.02, 0.98), \quad g = (0, 0.76), \quad o = (0.5, 0.5).$$

Update the cell elementwise:

$$c_t = (0.98(1) + 0.02(0),\ 0.02(0.5) + 0.98(0.76)) = (0.98, 0.7548).$$

Using the slide's approximate $\tanh(0.98) = 0.75$ and rounding 0.7548 to about 0.76, with $\tanh(0.76) \approx 0.64$,

$$h_t \approx (0.375, 0.32).$$

The first unit mostly keeps its old cell content and writes nothing; the second mostly erases its old content and writes the new positive candidate. Both expose about half the transformed cell content.

For the bonus head $W_y = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ -1 & -1 \end{bmatrix}$ and zero bias, scores are approximately $(0.375, 0.32, -0.695)$. Greedy prediction chooses a. These numbers are logits, not probabilities; softmax would preserve their ordering. Exact nonlinearities give slightly different decimals because the supplied lookup values are rounded.

5.1.7 GRUs, stacked networks and bidirectional processing

Teacher slides: 5.1. Sequence Models, pp. 21–24.

A gated recurrent unit (GRU) uses update and reset gates, with a candidate state, but no separate LSTM-style cell state. It interpolates between an old state and a proposed state. Different notations swap the roles of z and $1 - z$, so interpret the stated update equation before saying what a gate value near one means.

With three affine blocks and one bias per block, the cell count is $3H(D + H + 1)$, compared with $4H(D + H + 1)$ for the LSTM convention above. Fewer parameters does not guarantee better accuracy; it is an architectural trade-off.

Stacking recurrent layers adds depth: the lower layer's output at time t becomes an input to a higher layer. Each layer normally has its own weights. A bidirectional RNN processes a sequence in both directions and combines the resulting features. This gives access to future context within the observed input, but it cannot be used unchanged for online prediction when those future observations are unavailable.

For video, a CNN can encode individual frames and a recurrent model combine their temporal features. For captioning, an image or region representation conditions a word decoder. The task determines whether one final state, every state, or an output sequence is required.

5.1.8 Temporal convolutions

Teacher slides: 5.1. Sequence Models, pp. 25–27.

A temporal convolutional network (TCN) applies convolutions along time. A causal layer at position t uses only positions at or before t; the target alignment determines whether x_t itself is already known. A centered filter using future observations would leak information in a forecasting task.

Dilation increases the temporal span without increasing the number of filter coefficients. For stride-one layers of kernel width K and dilations $d_1, \ldots, d_L$, a simple sequential stack has receptive field

$$R = 1 + (K - 1) \sum_{\ell=1}^{L} d_\ell.$$

Two layers with $K = 3$, dilations 1 and 2, cover $1 + 2(1 + 2) = 7$ time positions. WaveNet illustrates stacked dilated causal convolutions. Training positions can be processed in parallel when all inputs are known, but autoregressive generation still depends on previously generated samples.

5.1.9 Encoder–decoder models and teacher forcing

Teacher slides: 5.1. Sequence Models, pp. 28–31.

An encoder reads the input sequence and produces a context representation. A decoder generates the target sequence, usually beginning with a start token and stopping at an end token. Input and output lengths need not be equal.

In basic recurrent seq2seq, the encoder's final state is a fixed-size context vector. This can become a bottleneck: information from a long input must be compressed into that one vector.

During **teacher forcing**, the decoder receives the true previous target token while learning to predict the next one:

$$L = -\sum_{t} \log p_\theta(y_t^* \mid y_{<t}^*, x).$$

At generation time, the true target is unavailable, so generated tokens are fed back. An early error can create contexts less familiar from training: exposure bias. Scheduled sampling mixes in model-generated inputs during training; it is an attempted remedy, not a guarantee.

Teacher forcing does not remove recurrence from an RNN decoder: h_t still depends on h_{t-1}. A nonrecurrent transformer decoder can instead compute training positions in parallel under a causal mask.

Attention reduces the fixed-context bottleneck

Keep all encoder states and form a different context for each output step:

$$c_t = \sum_{i=1}^{T} \alpha_{t,i} h_i, \qquad \alpha_{t,i} \geq 0, \quad \sum_i \alpha_{t,i} = 1.$$

The weights depend on the current decoding query and the encoded inputs. If two scalar values are 2 and 6, weights $(0.75, 0.25)$ produce context 3. Different output tokens can attend to different source positions. Attention can be added to an RNN; a transformer goes further by building its main sequence processing from attention blocks.

5.1.10 Revision questions and answers

Questions

1. Which changes during a forward pass: the recurrent weights or the hidden state?
2. How many parameters has an RNN with $D = 3, H = 2, O = 3$, including both biases?
3. Why does an unrolled RNN have more computation but not more parameters when T increases?
4. What does BPTT do with gradient contributions from repeated uses of a shared weight?
5. Does clipping fix vanishing gradients? Does truncated BPTT preserve arbitrarily long credit assignment?
6. What do $f_t \approx 1$ and $i_t \approx 0$ do to the LSTM cell?
7. Compute c_t for $c_{t-1} = 2, f_t = 0.5, i_t = 0.25, g_t = 0.8$.
8. How many cell parameters has an LSTM with $D = 3, H = 2$ and one bias per gate?
9. Does adding one vocabulary symbol necessarily increase the hidden-state dimension?
10. Why is a bidirectional RNN unsuitable for strict online forecasting?
11. What does teacher forcing supply to the decoder? What changes at inference?
12. What does attention change in a basic fixed-context seq2seq model?

Answers

1. The hidden state; weights are shared and fixed within the ordinary forward pass.
2. $6 + 4 + 2 + 6 + 3 = 21$.
3. More applications of the same transition are computed.
4. They are accumulated into the gradient of that one shared parameter.
5. No to both: clipping caps large gradients; truncation cuts long gradient paths.
6. Mostly preserves the old cell and writes little new content.
7. $1 + 0.2 = 1.2$.
8. $4(6 + 4 + 2) = 48$, excluding an output head.
9. No; input/output vocabulary dimensions change, while hidden width can stay fixed.
10. Its reverse-direction state depends on future observations.
11. The true previous token during training; generated tokens during inference.
12. Each output can access a weighted combination of all encoder states instead of relying solely on one fixed context.

5.1.11 Source guide

`5.1. Sequence Models.pdf`: sequence structure and vanilla RNNs (pp. 3–12), BPTT and gradient problems (pp. 13–16), LSTM/GRU and variants (pp. 17–24), TCNs (pp. 25–27), seq2seq and attention (pp. 28–32), and the character-model exercise (p. 33).

`5.1. Extra Material.pdf`: cell diagrams (pp. 2–4), sizes/counts (p. 5), parameters versus states (p. 6), and the worked numerical update (p. 7). These notes incorporate this extra deck into 5.1 rather than treating it as a different course topic.

5.2 Attention and transformers

5.2.1 Why attention?

Teacher slides: 5.2. Transformers, pp. 3–8.

An RNN transfers context through a sequence of hidden-state updates. A local convolution combines inputs within a receptive field. Attention lets an output directly combine information from selected input positions, with weights determined by the current data.

For captioning, the representation useful for one output word may differ from that needed for the next word. Rather than average all frame features once, attention computes a new weighted combination for each output query. A large weight means that a value vector contributes strongly to that particular output.

Full self-attention connects all allowed positions in one layer. Its short interaction paths help model distant relationships. Its dense pairwise computation is expensive for long inputs, and it supplies weaker built-in assumptions about spatial locality than a CNN.

Parallel training does not mean parallel generation

Transformer training can process known sequence positions together. Autoregressive generation still predicts a token, appends it, and repeats. Also, transformers have architectural assumptions and positional mechanisms: they are not models without any inductive bias.

5.2.2 Queries, keys and values

Teacher slides: 5.2. Transformers, pp. 9–14.

Use the slide's matrix convention: token embeddings are *rows* of $X \in \mathbb{R}^{N\times D}$. For one head:

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V,$$

with $W_Q, W_K \in \mathbb{R}^{D\times d_k}$ and $W_V \in \mathbb{R}^{D\times d_v}$. Biases may be included in an implementation; omit them unless a counting question specifies them.

Quantity	Role
Query q_n	What position n is looking for.
Key k_m	What position m offers for matching.
Value v_m	Information from position m to combine into the output.
Score $q_n k_m^\top$	Compatibility of one query with one key.
Weight a_{nm}	Normalized contribution of value m to output n.

Scaled dot-product attention is

$$S = \frac{QK^\top}{\sqrt{d_k}}, \qquad A = \mathrm{softmax}_{\mathrm{row}}(S), \qquad Z = AV.$$

For one row,

$$a_{nm} = \frac{\exp(s_{nm})}{\sum_j \exp(s_{nj})}, \qquad z_n = \sum_m a_{nm} v_m.$$

Weights are nonnegative and sum to one across the keys for each query. Columns need not sum to one, and A need not be symmetric. Different query/key projections and row normalization generally prevent symmetry.

Shapes are a useful error check

Quantity	Shape
X	$N \times D$
Q, K	$N \times d_k$
V	$N \times d_v$
S, A	$N \times N$
Z	$N \times d_v$

The dot product needs queries and keys of equal feature dimension. The value dimension can differ. For cross-attention with N_q queries and N_k source tokens, A is $N_q \times N_k$ and the output has N_q rows.

Two different meanings of weights

W_Q, W_K, W_V are learned parameters shared across positions. The attention coefficients A are computed from the current input. They change when the input changes, even when model parameters are fixed. Increasing N enlarges A, but does not enlarge these projection matrices.

Why divide by the square root?

Under the illustrative assumption of independent, zero-mean, unit-variance components, a query–key dot product has variance d_k. Dividing by $\sqrt{d_k}$ keeps its scale controlled. Otherwise large scores can make softmax extremely peaked, with small gradients. The division does not itself make the row sum to one; softmax performs that normalization.

5.2.3 Small attention calculations

Teacher slides: 5.2. Transformers, pp. 11–13.

The slide's weighted-sum example

Page 11 gives query $(1, 0)$, keys $(1, 0)$, $(0, 1)$, $(2, 1)$ and values $(1, 0)$, $(0, 2)$, $(3, 1)$. Its simplified, unscaled scores are $(1, 0, 2)$ and the given softmax weights are approximately $(0.245, 0.090, 0.665)$. Thus

$$z_1 = 0.245(1, 0) + 0.090(0, 2) + 0.665(3, 1) = (2.240, 0.845).$$

Input 3 contributes most strongly because its key has the highest score with this query. Other outputs reuse the keys/values but use their own queries.

This example intentionally uses the unscaled scores shown on p. 11. Standard scaled attention on p. 13 would divide by $\sqrt{2}$ first and produce different weights. Do not mix the two conventions within one calculation.

A two-position check

If a row's scores are equal and both positions are allowed, its weights are $(0.5, 0.5)$. With values $(2, 0)$ and $(0, 4)$, output is $(1, 2)$. If the second position is masked out, its weight is zero and the

output becomes $(2, 0)$.

Projection parameter count

For input width D, the three bias-free projections have

$$N_{\mathrm{QKV}} = 2Dd_k + Dd_v.$$

If $D = 4$ and $d_k = d_v = 2$, this is $3(4 \cdot 2) = 24$. Adding one bias per projected output component adds $2d_k + d_v = 6$. This count excludes an output projection, feed-forward network, embeddings and normalization.

5.2.4 Order: positional information

Teacher slides: 5.2. Transformers, pp. 15–17.

Unmasked self-attention without positional information is permutation-equivariant: permuting input rows permutes the output rows in the same way. It does not learn which token occupies which absolute position merely from the attention operation.

For example, the same occurrence of *man* among the same token multiset has no positional cue distinguishing who eats whom. Adding a position vector p_n to token embedding x_n changes this:

$$\widetilde{x}_n = x_n + p_n.$$

Mechanism	Main idea and limitation
Fixed sinusoids	Sine/cosine pairs at different frequencies encode position without learned table entries. Defined at new positions, but successful length extrapolation is not guaranteed.
Learned absolute positions	A table of $N_{\max}D$ learned values. Positions beyond the table need an extension strategy.
Relative positions	Attention incorporates positional offsets, emphasizing relationships between positions.
RoPE	Position-dependent rotations of queries/keys make their dot products depend on relative displacement as well as content.
ALiBi	A distance-dependent bias penalizes attention scores according to the head's rule.

The standard sinusoidal form is

$$p_{n,2j} = \sin\left(\frac{n}{10000^{2j/D}}\right), \qquad p_{n,2j+1} = \cos\left(\frac{n}{10000^{2j/D}}\right).$$

For the exam, understand its purpose and that it has no trainable position table; long trigonometric computations are unnecessary. A causal mask also changes the attention structure, but position encodings remain important in the architectures discussed here.

5.2.5 Tokenization, embeddings and context

Teacher slides: 5.2. Transformers, pp. 18–22.

Tokenization turns text into a sequence of vocabulary indices. A token need not be one word. Character tokenization uses a small vocabulary but long sequences; whole-word vocabularies struggle with unfamiliar words and inflections. Subword methods balance these issues by splitting uncommon words into smaller pieces.

Byte-pair encoding repeatedly merges frequent adjacent symbols during tokenizer construction. Once used for model training/inference, the vocabulary and tokenization rules define what the input indices mean.

A learned embedding table with V_{ocab} tokens and width D contains $V_{\text{ocab}}D$ parameters. Looking up a token index selects a row; a one-hot multiplication is an equivalent mathematical description, not a requirement to store large one-hot arrays.

The initial embedding of a token is context-independent. The token *bank* uses the same table entry in a river sentence and a money sentence. Its *contextual* representation after attention can differ because it combines information from different surrounding tokens.

For $V_{\text{ocab}} = 100, D = 8$, the table has 800 parameters. A sequence of length 20 produces a 20×8 embedding activation, not a 100×8 sequence activation. Vocabulary size, sequence length and feature width are different quantities.

5.2.6 Inside a transformer block

Teacher slides: 5.2. Transformers, pp. 23–29, 34.

Multi-head attention

Each head uses its own query/key/value projections. Its attention pattern can emphasize different relationships. Head outputs are concatenated and projected back to the model width:

$$\text{MHA}(X) = \text{Concat}(Z_1, \ldots, Z_h)W_O.$$

If each value head has width d_v, concatenation has width hd_v, and W_O has shape $hd_v \times D$. In the common choice $d_k = d_v = D/h$, the four combined bias-free projections contain $4D^2$ weights. Increasing the number of heads while keeping D fixed need not multiply this count; each head becomes narrower.

Residual connections and layer normalization

The slides show the original post-normalization order:

$$U = \text{LN}(X + \text{MHA}(X)), \qquad Y = \text{LN}(U + \text{FFN}(U)).$$

Residual addition requires matching shapes and provides a direct information/gradient path. Layer normalization typically normalizes each token across its feature coordinates:

$$\text{LN}(x)_j = \gamma_j \frac{x_j - \mu_x}{\sqrt{\sigma_x^2 + \epsilon}} + \beta_j.$$

Its statistics do not come from averaging across the mini-batch. It normally recomputes them from the token at both training and inference. The slide's shorthand comparison with batch normalization should not be read as saying they use the same axes or running statistics. Other transformer designs use pre-normalization; identify the order shown in a particular diagram.

The feed-forward sublayer

The same small MLP is applied independently at every token:

$$\mathrm{FFN}(x) = a(xW_1 + b_1)W_2 + b_2.$$

Attention mixes information between tokens; this MLP mixes feature coordinates within each token and adds nonlinear processing. It does not directly read another token.

With hidden width F, its count including biases is $DF + F + FD + D$. For $D = 4, F = 8$, that is $32 + 8 + 32 + 4 = 76$. Reusing the MLP at 10 or 100 positions does not change the count. Stacked transformer blocks, in contrast, ordinarily have different learned weights.

Readout

An encoder outputs one contextual vector per token. A token-level head can predict an entity label at each position. For a sequence-level task, a special CLS token can collect information through attention; a classifier then uses its final representation. CLS is a learned special input representation, not the answer label supplied in advance.

5.2.7 Three transformer families

Teacher slides: 5.2. Transformers, pp. 18, 28–35.

Family	Access to context	Typical use in the slides
Encoder-only	Bidirectional self-attention over observed input; no causal restriction.	BERT: contextual features, classification, retrieval, masked-token prediction.
Decoder-only	Causal self-attention over the prefix.	GPT-style autoregressive next-token generation.
Encoder–decoder	Encoder self-attention; decoder causal self-attention and cross-attention to the source.	Translation, captioning and speech-to-text; original Transformer, T5, Whisper.

An encoder can still use masks for padding or other restrictions. *No mask* in the family comparison means no *causal* mask, not that every implementation attends to padding.

Next-token prediction

An autoregressive model factorizes

$$p(x_1, \ldots, x_T) = \prod_{t=1}^{T} p(x_t \mid x_{<t}).$$

Its training objective is the negative log-probability of the true next tokens. With known training sequences, targets can be shifted by one position and many predictions computed in one pass. At generation, each newly sampled or selected token extends the prefix.

Causal masking

For an input token at position n predicting the next token, allow keys at positions $m \leq n$ and block $m > n$. Add $-\infty$ to blocked scores before softmax:

$$M_{nm} = \begin{cases} 0 & m \leq n, \\ -\infty & m > n. \end{cases} \qquad A = \mathrm{softmax}_{\mathrm{row}}(S + M).$$

Blocked weights become zero. Replacing a forbidden score by zero is not the same operation: $\exp(0) = 1$, so it could still receive weight.

For three positions, the allowed pattern including the diagonal is $\left[\begin{smallmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 1 & 1 & 1 \end{smallmatrix}\right]$. There are $N(N+1)/2$ allowed pairs, so masking alone still leaves quadratic growth. Causality prevents target leakage; it does not eliminate exposure bias from using true prefixes during training and generated prefixes at inference.

Cross-attention

Queries come from the decoder representation; keys and values come from the encoder output. An output token can therefore consult the whole observed source while still being forbidden from looking at future target tokens. With 3 target positions and 5 source positions, one cross-attention matrix is 3×5.

5.2.8 Computation and memory

Teacher slides: 5.2. Transformers, p. 36.

Dense attention forms N^2 scores per head and layer. Its pairwise operations cost order $N^2 d$ for fixed feature widths; a materialized score/weight matrix uses order N^2 memory. Doubling sequence length multiplies the number of entries by four, even though the projection parameters stay fixed.

For $N = 100$, one matrix has 10,000 entries. At four bytes each, this is 40,000 bytes before accounting for other activations, heads, layers and gradients. For $N = 200$, the same calculation gives 160,000 bytes.

During causal autoregressive generation with unchanged parameters, past keys and values can be cached. The new token needs new projections and attention to that cache. Cache size grows linearly with context length for fixed architecture, but each new query still has to process its allowed past context.

FlashAttention reorganizes exact attention computation into tiles to avoid materializing the full score matrix in high-bandwidth memory; it is not inherently a sparse approximation. Sparse or sliding-window attention changes which pairs interact. Grouped-query or multi-query attention shares key/value projections between query heads and reduces cache requirements. These are different efficiency mechanisms.

5.2.9 Vision transformers

Teacher slides: 5.2. Transformers, pp. 37–40.

Split an image into patches, flatten each patch and project it to an embedding. Add positional information, optionally prepend a CLS token, then apply transformer blocks.

For image height H, width W, channels C, and nonoverlapping square patches of side P dividing both dimensions:

$$N_{\text{patch}} = \frac{H}{P}\frac{W}{P}, \qquad D_{\text{patch}} = P^2C.$$

A biased linear projection to width D has $(P^2C)D + D$ parameters, shared across patches.

For a 32×32 RGB image and 8×8 patches, there are 16 patches, each with 192 scalar inputs. With $D = 12$, the projection has $192(12) + 12 = 2316$ parameters. Adding CLS gives 17 tokens and a 17×17 attention matrix per head. A learned absolute position table would add $17(12)$ values in this setup.

Halving patch side length at fixed image size gives four times as many patches and roughly sixteen times as many pairwise attention entries, ignoring CLS. Smaller patches preserve finer detail but increase cost. Hybrid models can use convolutional tokenization or hierarchical downsampling.

5.2.10 Worked classroom exercise: A or B?

Teacher slides: 5.2. Transformers, pp. 41–42.

The slide simplifies attention by setting $W_Q = W_K = W_V = W$ and replacing softmax normalization with division by 3. Its initial row embeddings are $e_{\text{eat}} = (0, 0, 1)$, $e_{\text{red}} = (0, 0, 0)$, $e_{\text{apple}} = (1, 1, 1)$, and

$$W_A = I_3, \qquad W_B = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 0 \end{bmatrix}.$$

Use the row-vector convention $Q = K = V = EW$ and the order (eat, red, apple). We calculate only the apple output.

For W_A, its query is $(1, 1, 1)$, scores against those three keys are $(1, 0, 3)$, and divided scores are $(1/3, 0, 1)$. Hence

$$z^A_{\text{apple}} = \tfrac{1}{3}(0, 0, 1) + 0(0, 0, 0) + 1(1, 1, 1) = (1, 1, 4/3).$$

For W_B, the projected values/keys are $(1, 0, 0)$, $(0, 0, 0)$ and $(2, 2, 1)$. The apple scores are $(2, 0, 9)$, yielding

$$z^B_{\text{apple}} = \tfrac{2}{3}(1, 0, 0) + 3(2, 2, 1) = (20/3, 6, 3).$$

Under the toy exercise's interpretation of the final axes as technology, nature and food, A emphasizes food in the eating context. This does not establish that A is universally a better learned projection: a general representation can change basis, and quality ultimately depends on an objective and evaluation.

This exercise is not standard normalized attention

Dividing scores by 3 does not force their sum to one or make them nonnegative in general. It replaces the normal softmax operation solely for the exercise. Also, using column-vector embeddings changes the order/transposes of the matrix multiplications; state a convention before calculating.

5.2.11 Revision questions and answers

Questions

1. Which determines attention weights: values, or query–key compatibility?
2. Must every column of the attention matrix sum to one?
3. For $N = 5, D = 8, d_k = 2, d_v = 3$, what are the shapes of Q, A and Z?
4. What is the purpose of dividing scores by $\sqrt{d_k}$?
5. With weights $(0.25, 0.75)$ and scalar values $(2, 6)$, what is the output?
6. Why does plain unmasked self-attention need positional information?
7. Do the MLP weights differ at every token position in one block?
8. Why must a next-token decoder mask future positions during training?
9. Where do queries, keys and values come from in encoder–decoder cross-attention?
10. Does doubling sequence length double the projection parameter count? What happens to N^2?
11. How many 4×4 patches are in a 16×16 image, and how many tokens with CLS?
12. Does a KV cache remove the need to generate successive tokens sequentially?

Answers

1. Query–key compatibility determines the weights; values supply the content being combined.
2. No. Each query row sums to one over its allowed keys.
3. $Q : 5 \times 2$, $A : 5 \times 5$, $Z : 5 \times 3$.
4. Control score scale as key dimension grows and reduce softmax saturation.
5. $0.25(2) + 0.75(6) = 5$.
6. Without positional cues it is permutation-equivariant and cannot distinguish positions by itself.
7. No; the same positionwise MLP is reused across tokens.
8. Those future tokens contain prediction targets and would leak the answers.
9. Queries from the decoder; keys and values from encoder outputs.
10. Projection count stays fixed; attention matrix entries multiply by four.
11. 16 patches and 17 tokens with CLS.
12. No; it reuses previous computations, while each new token still depends on the generated prefix.

5.2.12 Source guide and connection to Lab 5

`5.2. Transformers.pdf`: motivation (pp. 3–8), attention equations and examples (pp. 9–14), positions (pp. 15–17), tokenization/embeddings (pp. 18–22), blocks and encoders (pp. 23–29), decoding/masks/cross-attention (pp. 30–35), efficiency (p. 36), vision transformers (pp. 37–40), and the A/B exercise (pp. 41–42).

For the Lab 5 connection, be able to explain what each matrix represents, which quantities are learned versus computed, how context changes a token representation, and why masks change which information is available. Correct dimensions and a clear interpretation are more useful than memorizing a large matrix multiplication.

5.3 Foundation models

5.3.1 What makes a model a foundation model?

Teacher slides: 5.3. Foundation models, pp. 9–13.

A foundation model is trained on broad data at scale and can be adapted to many downstream tasks. The key idea is **transfer learning at scale**: an expensive shared pretraining stage produces reusable representations or behaviors, instead of training a separate model from scratch for every task.

A transformer is an architecture; a foundation model describes the scope of training and reuse. The terms are not synonyms. A small transformer trained only for one narrow dataset is not automatically a foundation model. Likewise, a foundation model can be useful as an encoder without being a conversational or image-generation system.

Broad pretraining can provide useful features, language patterns and cross-modal associations. Performance still depends on the downstream task, domain, data quality and adaptation. Broad training does not guarantee accurate answers on every new problem.

Identify three things

For any named model, ask: what input does it accept, what objective was used to train it, and what output does it produce? A feature vector, a token distribution, a mask and a generated image are different outputs, even if the models share transformer components.

5.3.2 Architecture, objective and adaptation

Teacher slides: 5.3. Foundation models, pp. 5–8, 15–17.

Family	Training/attention idea	Useful output
BERT-style encoder	Bidirectional input context; predict hidden tokens. Original BERT also used next-sentence prediction.	Contextual token features or a sequence representation for a task head.
GPT-style decoder	Causal self-attention; predict the next token from the prefix.	Next-token probabilities, used repeatedly for generation.
Encoder–decoder	Encode a source and generate a target with causal self-attention plus cross-attention.	A conditional sequence, such as a translation or transcription.

BERT-style masked-token prediction learns from both sides of a missing token. GPT-style next-token prediction cannot use the unknown continuation. Both can obtain training targets from text itself, even though their information restrictions differ.

The slides use named model families and historical parameter counts to illustrate these ideas. Treat those as course examples, not a specification of every version or a ranking of current products. Detailed internals of some proprietary models are not publicly established.

Ways to reuse a pretrained model

Method	What changes?
Frozen feature extraction	Use the representation as input to a new task head; keep the backbone fixed.
Fine-tuning	Update some or all pretrained parameters on the downstream training objective.
Prompting	Supply instructions or context as input; ordinary inference leaves weights fixed.
Few-shot prompting	Include several task examples in the prompt; the model conditions on them without a gradient update.
Zero-shot use	Apply the model to a task/category without task-specific example training in the stated setup.

The meaning of zero-shot depends on the evaluation protocol. It does not mean the pretraining data necessarily contained nothing related to the category. In-context examples change the current computation and context, not ordinarily the stored parameters.

5.3.3 Where the training signal comes from

Teacher slides: 5.3. Foundation models, pp. 13–14; 5.4. Unsupervised learning, pp. 2, 24.

The slide pipeline separates several sources of supervision:

Stage	Training signal	Interpretation
Self-supervised pretraining	Targets constructed from observed data, such as held-out or next tokens.	Learn broad structure without manually labeling every prediction target.
Supervised fine-tuning (SFT)	Demonstrated desired responses or labeled task examples.	Learn to imitate appropriate outputs for prompts/tasks.
Preference-based RL	Preferences used to train or supply a reward signal.	Sample responses and adjust behavior to favor rewarded ones.
Verifiable RL	Automatic checkers, such as answer matching or tests.	Reward outputs according to an externally evaluated criterion.

These are complementary mechanisms. A vision-language model (VLM) can combine image/text pretraining with instruction examples and later reward-based training. One model's complete training history need not fit exactly one supervision label.

SFT versus preference optimization

SFT treats a desired response as a target sequence and increases its likelihood. A preference comparison says that one candidate response is better than another; it need not provide a perfect response token by token. In a common RLHF setup, a reward model learns such preferences and an optimization procedure increases expected reward.

Not every preference-based method explicitly runs an RL rollout-and-update loop. The conceptual point is the difference between imitating a demonstrated target and optimizing a preference-derived signal, not that every named post-training recipe has identical machinery.

Verifiable rewards

A checker may determine whether a mathematical answer matches a reference, whether generated code passes tests, or how well a predicted box overlaps a target. These signals can provide scalable feedback. The reward only measures what the checker actually checks; passing incomplete tests does not establish correctness in every possible case.

Alignment is not a guarantee of factual correctness

The slides mention reducing hallucinations through post-training. Reward optimization can improve measured behavior, but a high preference score or fluent answer does not prove truth. A learned reward can reflect annotator biases, and an automatic verifier can be incomplete or flawed.

A simple reward calculation

Suppose a policy chooses response A with probability 0.7 and response B with probability 0.3, and a checker assigns rewards 1 and 0. Expected reward is $0.7(1) + 0.3(0) = 0.7$. Increasing A's probability to 0.9 raises it to 0.9. This is a toy illustration of expected reward, not a complete policy-gradient update.

5.3.4 Scale, storage and mixture of experts

Teacher slides: 5.3. Foundation models, pp. 12, 17, 26.

Parameter count is the number of learned scalar coefficients. Storage also depends on precision. For raw weights only,

$$\text{bytes} = (\text{parameter count}) \times \frac{\text{bits per stored parameter}}{8}.$$

A hypothetical 100-million-parameter model needs about 400 MB at 32 bits or 200 MB at 16 bits, using decimal MB. These estimates exclude metadata, activations, gradients, optimizer state and inference caches.

In a sparse mixture-of-experts (MoE) model, a router selects a subset of expert subnetworks for each token. Some components are shared and used for every token. Distinguish:

- **Total parameters**: all shared and expert parameters stored by the full model.
- **Active parameters**: the subset used along a given token's computation.

A toy model with 40 million shared parameters and 16 experts of 20 million each has 360 million total parameters. If two experts are used per token, that token uses $40 + 2(20) = 80$ million parameters, about 22.2% of the total.

This ratio is not a guaranteed speedup. Routing, communication, memory access and the operations in the active path also matter. Different tokens can select different experts, so the full set may be used across a batch. Sparse activation reduces some computation without making the unused experts disappear from model storage.

5.3.5 CLIP: align images and text

Teacher slides: 5.3. Foundation models, pp. 19–20.

CLIP uses an image encoder and a text encoder to map paired images and descriptions into a shared representation space. Training increases the similarity of matching image–text pairs relative to mismatched pairs.

With B paired examples in a batch, the similarity matrix has $B \times B$ entries. The diagonal represents matched pairs under aligned ordering; the off-diagonal entries provide mismatches. Both image-to-text and text-to-image matching can provide training signals.

Zero-shot classification

To classify an image among candidate categories:

1. Write a text description for each candidate category.
2. Encode the image and the candidate texts.
3. Compare image/text embeddings, often after normalization.
4. Select the category whose description has the largest similarity.

This supplies a classification rule without fitting a new labeled classifier for those categories. Performance can depend on wording and which candidate labels are offered.

For unit-normalized vectors, cosine similarity is their dot product. If the image vector is $(1, 0)$ and candidate vectors are $(0.8, 0.6)$ and $(0, 1)$, similarities are 0.8 and 0. The first is the better match among these candidates. A similarity of 0.8 is not automatically an 80% calibrated probability of correctness.

An embedding model is not an image generator

CLIP supplies aligned representations and similarity scores. It can guide or provide conditioning/evaluation in a larger generative system, but the image encoder/text encoder pair alone does not decode a new image from a prompt.

5.3.6 DINOv2 and SAM

Teacher slides: 5.3. Foundation models, pp. 21–22.

DINOv2: reusable visual features

DINOv2 is a self-supervised visual-representation example. A student learns from a teacher using different views of images without requiring semantic class labels or text captions for the pretraining objective. Image-level and patch-level information support reusable visual features.

The teacher is part of the training mechanism, not a human annotator supplying class names. Learned features can then support image or pixel tasks with an appropriate readout. The slide's broad transfer claim should not be interpreted as a guarantee that every task works perfectly without adaptation or a task-specific head.

SAM: promptable segmentation

The Segment Anything Model illustrates a foundation model for segmentation. It combines an image with a prompt specifying what to segment, such as points or a box in the original setting, and produces mask predictions.

A segmentation mask identifies image regions. It is not the same output as an image-classification label or a newly generated photograph. Promptable segmentation still requires deciding what object/region to request and evaluating whether the predicted boundary is useful. The exam-level distinction is the task and output; version-specific feature lists are not needed to explain that mechanism.

Example	Learned capability	Main output/use
CLIP	Image–text alignment from paired data.	Similarity/retrieval and zero-shot classification.
DINOv2	Visual representation from self-supervised image views.	Features for downstream image/pixel tasks.
SAM	Prompt-conditioned segmentation.	Object/region masks.

5.3.7 Language-conditioned image generation

Teacher slides: 5.3. Foundation models, pp. 23–25.

A text-to-image system can combine distinct components:

$$\text{text prompt} \longrightarrow \text{text features} \longrightarrow \text{conditioned denoising} \longrightarrow \text{image decoding}.$$

In a latent-diffusion example, a text encoder represents the prompt, a denoiser iteratively transforms a noisy latent while conditioning on the prompt, and a decoder maps the final latent to an image. The denoiser may use a U-Net or a transformer, depending on the model.

Next-token language prediction and continuous denoising are different objectives. Using language features to condition diffusion does not mean the image generator is simply selecting the next text token. Conversely, not every generative model must use diffusion. Topic 5.5 explains the broader distinctions.

5.3.8 Benefits and limitations to explain

Teacher slides: 5.3. Foundation models, pp. 10–14, 26.

Reusable representations can reduce task-specific training effort, allow transfer with fewer labels, and support multiple modalities. Large-scale training and deployment can require substantial memory, computation and data preparation.

Common pretrained components can also propagate biases and failure patterns across many downstream systems. Evaluate the actual task, relevant subgroups and deployment distribution. Explainability concerns ask what evidence a prediction used; fairness concerns ask how performance and impacts are distributed. Neither is settled solely by a model's size or overall accuracy.

These are conceptual issues raised by the course, not a legal checklist. For a short exam answer, connect a concrete property to a consequence: for example, a generic pretraining dataset may poorly represent a specialized engineering domain, so downstream validation is still necessary.

5.3.9 Revision questions and answers

Questions

1. Is every transformer a foundation model?
2. What distinguishes pretraining from downstream fine-tuning?
3. Does ordinary few-shot prompting update the model's weights?
4. Where do next-token pretraining targets come from?
5. How does a preference ranking differ from an SFT demonstration?
6. What does verifiable RL use as its reward source?
7. Can a model have many more total than active parameters?
8. What are CLIP's inputs and what does its similarity compare?
9. Is DINOv2's teacher necessarily a human supplying class labels?
10. What does SAM produce?
11. Does a text encoder alone implement diffusion image generation?
12. How many decimal MB store 50 million weights at 16 bits, excluding overhead?

Answers

1. No. Transformer is an architecture; foundation-model status concerns broad training and adaptability.
2. Pretraining learns a broadly reusable base; fine-tuning adapts parameters to a later task or behavior.
3. No. It supplies examples as input context.
4. The observed sequence itself: the following token is the target.
5. A ranking compares candidate outputs; SFT supplies a desired output to imitate.
6. An automatic checker or other verifiable evaluation criterion.
7. Yes, as in sparse MoE routing; all experts remain part of the total model.
8. Images and text; similarity compares their embeddings in a shared space.
9. No. It is a model within a self-supervised teacher–student procedure.
10. Prompt-conditioned segmentation masks.
11. No. A denoising/generative component and, in latent systems, an image decoder are also needed.
12. $50 \times 10^6 \times 2 = 100 \times 10^6$ bytes, or 100 MB.

5.3.10 Source guide

`5.3. Foundation models.pdf`: transformer-family recap (pp. 5–8), definition/scale and training signals (pp. 9–14), language/multimodal examples (pp. 15–18), CLIP/DINOv2/SAM (pp. 19–22), image generation (pp. 23–25), and efficiency/explainability/fairness questions (p. 26). Administrative pages are not study theory. Numerical storage, reward and MoE examples here are explanatory applications rather than specifications of named commercial systems.

5.4 Unsupervised and self-supervised learning

5.4.1 Learning without task labels

Teacher slides: 5.4. Unsupervised learning, pp. 2–8, 23–25.

In supervised learning, training examples include a target for the task, such as a class, a numerical value or a segmentation map. Unsupervised learning seeks structure without those task labels: groups, low-dimensional representations, distributions or other patterns.

The absence of human class labels does not mean the absence of an objective. A clustering objective can reward compact groups; a reconstruction objective can compare an output with its input; a self-supervised prediction task can derive targets from the data itself.

Setting	Training signal	Example
Supervised	Provided task targets.	Image with a known object class.
Unsupervised	Structure/objective without task labels.	Cluster feature vectors.
Self-supervised	Targets constructed from the data or transformations.	Predict hidden tokens or a rotation applied to an image.
Semi-supervised	A mixture of labeled and unlabeled examples.	Learn from a small labeled set plus consistency or pseudo-labels on more images.
Reinforcement learning	Rewards from actions and outcomes.	Improve a policy using interaction feedback.

Self-supervised learning is often grouped under unsupervised representation learning, but its optimization can look like supervised prediction once its targets have been constructed. Deep RL uses neural networks to represent policies, values or models; it remains reward-based learning, rather than becoming unsupervised merely because it uses a deep network.

Why unlabeled examples can help

Labels may require expertise, time or dense annotation. Additional unlabeled examples can reveal which inputs occur frequently and where the data has structure. However, those examples should be relevant to the target task and distribution.

If unlabeled data comes from an unrelated domain or reinforces incorrect pseudo-labels, it can harm performance. A cluster boundary is not automatically the correct class boundary. Validate the chosen objective and assumptions rather than assuming that more unlabeled data must improve the result.

5.4.2 PCA: compress along directions of variation

Teacher slides: 5.4. Unsupervised learning, pp. 9–11, 36.

Principal component analysis finds orthogonal directions of variation. After centering samples by their mean μ, the leading eigenvectors of the covariance matrix define the directions with largest variance.

For a column-vector sample $x \in \mathbb{R}^D$, collect the first k orthonormal directions in $U_k \in \mathbb{R}^{D\times k}$:

$$z = U_k^\top (x - \mu), \qquad \widehat{x} = \mu + U_k z.$$

The first equation is a reusable mapping to k dimensions. The second reconstructs the part retained in that subspace.

What PCA optimizes

Among centered orthogonal linear projections of the same rank, PCA maximizes retained variance and minimizes total squared reconstruction error on the fitted data. This does not mean it preserves every kind of information, the most useful class signal, or nonlinear structure.

Explained variance

Let covariance eigenvalues be $\lambda_1 \geq \cdots \geq \lambda_D \geq 0$. Retaining k components preserves the variance fraction

$$\frac{\sum_{j=1}^{k} \lambda_j}{\sum_{j=1}^{D} \lambda_j}.$$

If the eigenvalues are $(6, 3, 1)$, the first component preserves 60% and the first two preserve 90%. Discarding the third loses 10% of total variance. These are not percentages of classification accuracy.

For a simple projection, let $\mu = (1, 1)$, first component $u = (1, 0)$ and sample $x = (4, 3)$. Then $z = u^\top (x - \mu) = 3$, reconstruction is $(4, 1)$, and squared reconstruction error is $0^2 + 2^2 = 4$.

Practical interpretation

Features with large numerical scale can dominate variance. Centering and, when appropriate, standardization therefore affect PCA. Fit such preprocessing and the PCA directions on the training split; fitting them on held-out test data leaks information about that set into the representation.

PCA can compress neural-network features or remove some low-variance variation. A low-variance direction can still be essential for a target task, so the downstream validation result matters. The slide's phrase about minimizing lost information is specifically justified by the squared-error/variance criterion, not a universal semantic notion of information.

5.4.3 t-SNE: visualize local neighborhoods

Teacher slides: 5.4. Unsupervised learning, pp. 12–18, 36.

t-SNE places the supplied high-dimensional samples in two or three dimensions while trying to preserve local similarities. Nearby points in the original representation are encouraged to remain neighbors in the visualization.

The output axes are coordinates of the visualization; they are not ordered principal components with explained-variance percentages. Widely separated groups, apparent cluster sizes and empty gaps should not be interpreted directly as faithful global distances or densities in the original space.

Perplexity and interpretation

Perplexity controls the effective neighborhood scale used when converting distances into similarities. A small value emphasizes more local structure; larger values consider broader neighborhoods. It is not a requested number of clusters. Initialization, optimization and other settings can also change the picture.

The slides show a commonly explored range of 5–50, but it is a practical example range rather than a guarantee for every dataset size. A convincing-looking plot should be checked across reasonable settings and against the original representation/task.

PCA versus t-SNE

Property	PCA	Standard t-SNE
Mapping	Learned linear projection.	Coordinates optimized for the supplied dataset.
Main focus	Variance and squared reconstruction error.	Local neighborhood similarities.
New points	Apply the fitted projection.	No simple fitted linear map in the standard formulation; extensions exist.
Typical use here	Compression/preprocessing and visualization.	Exploratory 2D/3D visualization.
Interpretation	Components ordered by variance.	Plot axes and global gaps lack PCA-style interpretation.

The AlexNet example uses 4096-dimensional feature vectors. Plotting them after t-SNE can reveal possible organization, but a separated plot alone does not prove that a classifier generalizes or that the discovered groups equal semantic classes.

5.4.4 K-means: assignments and means

Teacher slides: 5.4. Unsupervised learning, pp. 19–20, 36.

Given K clusters, standard k-means minimizes within-cluster squared Euclidean distance:

$$J = \sum_{i=1}^{n} \|x_i - \mu_{c_i}\|^2.$$

Its alternating steps are:

1. Assign each sample to the nearest current centroid.
2. Replace each centroid with the mean of its assigned samples.
3. Repeat until a stopping criterion is met.

The mean is the appropriate update for this squared-distance objective. Substituting an arbitrary distance does not automatically preserve the same update rule or algorithm.

One small iteration

Take scalar samples $\{1, 2, 8, 9\}$, $K = 2$, and initial centers 1 and 8. Nearest-center assignment produces groups $\{1, 2\}$ and $\{8, 9\}$. New centers are

$$\mu_1 = (1 + 2)/2 = 1.5, \qquad \mu_2 = (8 + 9)/2 = 8.5.$$

Their squared-distance objective is $4(0.5^2) = 1$. Cluster indices 1 and 2 have no intrinsic class meaning; swapping their names leaves the grouping unchanged.

What a result does and does not mean

Initialization can lead to different local optima. K is chosen externally, not automatically discovered by the standard algorithm. Feature scaling changes distances. Standard k-means is most naturally suited to compact groups in its chosen geometry and can struggle with elongated, irregular or very unequal-density groups.

Increasing K can reduce the optimal training distortion simply by allowing more centers. That does not establish that the resulting clustering is more meaningful. Evaluate stability, interpretability and downstream usefulness as appropriate.

5.4.5 Hierarchical clustering and topic mixtures

Teacher slides: 5.4. Unsupervised learning, pp. 21–22.

Hierarchical structure

Agglomerative clustering begins with each point as its own cluster and repeatedly merges clusters. Divisive clustering begins with all points together and repeatedly splits groups. A dendrogram records this hierarchy.

The result depends on how distances between clusters are defined. For example, single linkage uses the closest cross-cluster pair; complete linkage uses the farthest; average linkage averages distances. Choosing a cut level in the hierarchy determines the final number of groups. A hierarchy supplies structure across resolutions rather than automatically identifying one uniquely correct partition.

LDA here means Latent Dirichlet Allocation

The slide explicitly uses LDA for **Latent Dirichlet Allocation**, a probabilistic topic model, not Linear Discriminant Analysis.

Each document has a mixture of topics, and each topic has a distribution over words. A document about sports clothing might combine sports and fashion topics. This differs from a hard clustering model that assigns the entire document to exactly one group.

If a document's topic mixture is $(0.6, 0.4)$ and a particular word has probabilities $(0.2, 0.05)$ in those topics, its mixture probability is

$$p(w \mid d) = 0.6(0.2) + 0.4(0.05) = 0.14.$$

Topic proportions are probabilities over topics, not a list of words with binary membership. Topic names are interpretations assigned to learned word patterns rather than labels required during fitting.

5.4.6 Self-supervision through proxy tasks

Teacher slides: 5.4. Unsupervised learning, pp. 24–27, 35.

A proxy or pretext task constructs a prediction target that can be obtained automatically. The aim is often to learn a representation useful for a later task.

Proxy task	Automatically available target	Useful structure encouraged
Rotation prediction	The rotation applied to the image.	Orientation and object appearance.
Relative patch position	Where sampled patches came from.	Spatial relationships and scene layout.
Jigsaw reconstruction	The original patch arrangement.	Parts and their configuration.
Colorization	Removed color information.	Context and plausible object appearance.
Inpainting/masked prediction	The hidden original content.	Dependencies between visible and missing regions.

Targets exist even though a human did not annotate a class for every training item. After pretraining, a proxy-specific head may be discarded while retaining the encoder.

Success on the proxy objective is not identical to success on a downstream task. Shortcuts can solve a proxy without learning the intended semantic structure. For example, visible image-processing artifacts might reveal a transformation. Evaluation on the intended downstream task is therefore necessary.

5.4.7 Contrastive representations

Teacher slides: 5.4. Unsupervised learning, pp. 28–29, 35.

Contrastive learning encourages related inputs to have similar representations and contrasts them with other inputs. Positive pairs can be two augmented views of the same image or naturally paired modalities. Negative pairs supply examples that should be distinguished under the chosen objective.

The choice of augmentation expresses an invariance assumption. Two crops may preserve object identity; an aggressive crop may remove the object. A color change may be harmless for some tasks and destroy label-relevant information for others.

A simple triplet illustration

For anchor a, positive p, negative n and margin m, an illustrative distance-based triplet loss is

$$L = \max(0, d(a,p) - d(a,n) + m).$$

If $d(a,p) = 0.4$, $d(a,n) = 0.7$ and $m = 0.5$, then $L = 0.2$. Zero loss requires the negative to be at least m farther from the anchor than the positive, under this convention.

This illustrates the slide's triplet idea; other contrastive losses use normalized similarities and many negatives. A negative may be a different image of the same semantic class, so pair construction is not automatically identical to true class supervision.

Representation collapse

If every input is mapped to one constant vector, positive pairs match perfectly but the representation conveys no distinctions. This is collapse. Negatives can discourage it, but there are also successful methods without explicit negatives that use other asymmetries and learning mechanisms.

5.4.8 BYOL, SuperPoint and semi-supervised extensions

Teacher slides: 5.4. Unsupervised learning, pp. 30–34.

BYOL

Bootstrap Your Own Latent (BYOL) uses two augmented views and an online/target network pair. The online branch, including a predictor, is trained to match the target representation of the other view. Gradients do not backpropagate through the target branch; target parameters track the online parameters through a moving average.

A simplified teacher update is

$$\theta_{\text{target}} \leftarrow \tau\theta_{\text{target}} + (1-\tau)\theta_{\text{online}}.$$

For scalar values 2 and 4 and $\tau = 0.9$, the new target value is 2.2. This is an explanatory calculation of the averaging mechanism.

BYOL has no explicit negative pairs in its basic objective. It is more precise to call it self-distillation or non-contrastive representation learning than to assume every such method separates negatives. Its predictor, stop-gradient target and update design matter; simply minimizing distance between arbitrary identical networks is not the same training procedure.

After representation learning, use the encoder's learned features for a downstream task; the whole two-branch training construction need not remain in the deployed model.

SuperPoint

The slides show interest-point detection and description learned using synthetic supervision and transformed images, then improved through generated labels. Known geometric transformations, such as homographies, help construct consistency relationships without manually labeling every real-image correspondence.

A detector predicts useful image locations; a descriptor provides a feature vector for matching those locations. Matching with a separate method is a further component, not the same task as learning the point detector.

Pseudo-labels, teachers and memory banks

Semi-supervised systems can use a model's predictions as training targets on unlabeled examples, often with confidence or consistency criteria. These pseudo-labels are estimates, not guaranteed truth. A teacher can produce more stable targets; a memory bank can retain representative features used in comparisons.

The slide's pixel-level example encourages same-class pixel representations to be similar across images while using both labeled and unlabeled data. A memory bank stores selected feature

information, not necessarily all raw training images. Such mechanisms can propagate useful structure, but incorrect pseudo-labels can reinforce mistakes.

5.4.9 Revision questions and answers

Questions

1. Can self-supervised training have explicit targets and a loss?
2. Does unlabeled data from any distribution necessarily help a supervised task?
3. With PCA eigenvalues $(6, 3, 1)$, what fraction of variance do the top two retain?
4. Does retaining 90% of variance guarantee 90% classification accuracy?
5. Is t-SNE perplexity a requested cluster count?
6. Can a large gap in a t-SNE plot be read as a reliable global distance?
7. One k-means cluster contains scalar points 2, 4 and 9. What is its updated center?
8. What distinguishes agglomerative from divisive clustering?
9. What does LDA stand for in these slides, and may a document have several topics?
10. What are positive pairs and what does representation collapse mean?
11. Does basic BYOL require explicit negatives?
12. Why can pseudo-labeling amplify an error?

Answers

1. Yes; targets can be derived automatically from inputs or transformations.
2. No. Distribution relevance and method assumptions matter.
3. $(6 + 3)/10 = 0.9$.
4. No; variance and predictive label information are different quantities.
5. No; it controls an effective neighborhood scale.
6. No; t-SNE primarily aims to preserve local relationships.
7. $(2 + 4 + 9)/3 = 5$.
8. Agglomerative merges initially separate groups; divisive splits an initial whole.
9. Latent Dirichlet Allocation; each document can be a mixture of topics.
10. Positives are paired inputs to align; collapse maps different inputs to indistinguishable representations.
11. No. It uses an online predictor and a target network with stop-gradient and moving-average updates.
12. An incorrect prediction becomes a training target and can be reinforced.

5.4.10 Source guide

`5.4. Unsupervised learning.pdf`: supervision strategies and motivation (pp. 2–8), PCA (pp. 9–11), t-SNE and feature visualization (pp. 12–18), k-means/hierarchy/LDA (pp. 19–22), proxy tasks (pp. 24–27), contrastive representations (pp. 28–29), BYOL (pp. 30–31), SuperPoint and semi-supervised examples (pp. 32–34), and summaries (pp. 35–36). The numerical projection, clustering, mixture, triplet and teacher-update examples are study calculations derived from these concepts.

5.5 Generative models

5.5.1 Discriminative and generative modeling

Teacher slides: 5.5. Generative Models, pp. 3–6.

A discriminative classifier models a decision rule or a conditional distribution such as $p(y \mid x)$: given an observation, predict its label. A generative model represents how observations can arise, for example through $p(x)$, $p(x \mid y)$ or a joint distribution $p(x, y)$.

In the classical classification formulation, a generative approach can model class priors $p(y)$ and likelihoods $p(x \mid y)$, then use Bayes' rule to obtain $p(y \mid x)$. Modern generative models can also model observations without class labels at all.

Training data supplies samples from an unknown distribution. The aim is to learn enough structure to sample plausible new observations. Memorizing a few training examples is not the same as adequately modeling the distribution's diversity.

Generation does not require an explicit tractable density

Some models define tractable probabilities; others generate samples without providing an easily evaluated likelihood. A GAN generator is a key example of the latter. The general capability to generate samples should not be confused with having a closed-form density for every input.

An autoregressive language model is also generative: it models a sequence through successive token probabilities. This chapter emphasizes image-oriented examples, but generation applies to text, sound, video and other data.

5.5.2 Autoencoders: learn to reconstruct

Teacher slides: 5.5. Generative Models, pp. 7–8.

An autoencoder has an encoder f_θ and decoder g_ϕ:

$$z = f_\theta(x), \qquad \widehat{x} = g_\phi(z).$$

It is trained so that $\widehat{x}$ resembles x, for example by minimizing mean squared reconstruction error:

$$L_{\text{rec}} = \frac{1}{D} \sum_{j=1}^{D} (x_j - \widehat{x}_j)^2.$$

The target is the original input, so no externally supplied semantic class label is needed. A low-dimensional bottleneck or other restrictions encourage a useful representation rather than unrestricted copying.

Why the bottleneck matters

With enough unconstrained capacity, an autoencoder can learn an identity-like mapping with little useful compression. Restricting latent dimension, adding regularization or using a denoising task can encourage structure. A compressed representation is not automatically the best representation for every downstream task.

An encoder–decoder for segmentation also maps through intermediate features, but its target is a semantic mask rather than the input image. Similar architecture does not imply the same objective or supervision.

Small reconstruction and count examples

For $x = (1, 3)$ and $\widehat{x} = (2, 1)$, the sum of squared errors is $1 + 4 = 5$ and the mean is $5/2 = 2.5$. State whether the loss uses a sum or mean.

A dense autoencoder $8 \to 3 \to 8$ with one bias per unit has

$$3(8 + 1) + 8(3 + 1) = 27 + 32 = 59$$

parameters. Its latent vector has three entries, but its learned model has 59 coefficients. Latent dimension, parameter count and number of training samples are different quantities.

Why arbitrary latent sampling can fail

A deterministic autoencoder learns to decode the latent codes that its encoder produces on the data. It does not necessarily arrange the whole latent space to match a simple sampling distribution. Random Gaussian latent vectors may land in poorly trained regions and decode badly.

The decoder can mathematically accept any latent vector of the correct shape. The slide's warning is about output quality outside the learned code distribution, not that decoding those vectors is computationally impossible.

5.5.3 Variational autoencoders

Teacher slides: 5.5. Generative Models, p. 8.

A variational autoencoder (VAE) encodes an input into a distribution over latent vectors, often a diagonal Gaussian:

$$q_\phi(z \mid x) = \mathcal{N}\Big(\mu_\phi(x), \operatorname{diag}(\sigma_\phi(x)^2)\Big) .$$

The encoder predicts mean and scale parameters, not a single fixed code. A decoder models observations conditional on a sampled latent. A common prior is $p(z) = \mathcal{N}(0, I)$.

Reconstruction plus regularization

A standard negative-ELBO objective has the form

$$L(x) = \underbrace{\mathrm{E}_{q_\phi(z|x)}[-\log p_\theta(x \mid z)]}_{\text{reconstruction term}} + \underbrace{D_{\mathrm{KL}}(q_\phi(z \mid x) \| p(z))}_{\text{prior regularization}} .$$

The reconstruction term rewards retaining information needed to reproduce the input. The KL term penalizes deviations of each encoded distribution from the prior, encouraging a latent space compatible with sampling. Squared-error reconstruction corresponds to a particular Gaussian observation model and scaling, rather than being the only possible reconstruction loss.

Increasing the relative weight of the KL term changes a trade-off; it does not guarantee improvements in both reconstruction and sampling. Excessive pressure can make the latent carry little information about the input.

Reparameterization

Write a sample as

$$\epsilon \sim \mathcal{N}(0, I), \qquad z = \mu + \sigma \odot \epsilon.$$

Randomness is supplied by ϵ, while the path through μ and σ is differentiable. For one coordinate with $\mu = 1, \sigma = 2, \epsilon = -0.5$, the sampled latent is $z = 0$.

A small KL calculation

For a diagonal Gaussian versus $\mathcal{N}(0, I)$,

$$D_{\mathrm{KL}} = \frac{1}{2} \sum_j \left(\mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1 \right).$$

For one coordinate with $\mu = 1, \sigma = 1$, this is $\frac{1}{2}(1 + 1 - 0 - 1) = 0.5$ nats. With $\mu = 0, \sigma = 1$, it is zero because the two distributions match. Here σ is a standard deviation; σ^2 is a variance.

During reconstruction, sample from the encoder's distribution for a given input. During unconditional generation, sample z from the prior and decode it. A structured latent objective makes this sampling procedure meaningful, but does not guarantee realistic outputs for every point in the infinite prior support.

5.5.4 GANs: a learned adversarial signal

Teacher slides: 5.5. Generative Models, pp. 9–13.

A generative adversarial network trains two networks:

- The generator G maps a random input z to a synthetic sample $G(z)$.
- The discriminator D learns to distinguish real training examples from generated ones.

The discriminator supplies a learned signal about realism. The generator adjusts its outputs so the discriminator is more likely to regard them as real.

In the original minimax formulation,

$$\min_G \max_D \left[\mathrm{E}_{x \sim p_{\mathrm{data}}} \log D(x) + \mathrm{E}_{z \sim p(z)} \log(1 - D(G(z))) \right].$$

Read the roles rather than memorizing a long derivation: D seeks high scores for real samples and low scores for fake samples, while G changes the fake samples to oppose that distinction.

Alternating optimization

When updating D, treat generated samples as examples labeled fake and real data as examples labeled real. When updating G, hold the discriminator's parameters fixed but differentiate through its computation to the generated image and generator.

Fixed discriminator parameters do not mean no gradient can pass through the discriminator's input. That gradient is precisely how the generator can learn.

The real/fake labels are constructed from the data source, not human semantic labels such as dog or cat. Thus the slide's description as weak supervision should not be taken to require manually labeled object categories for an unconditional GAN.

Generator losses and interpretation

A commonly used non-saturating generator loss is

$$L_G = -\,\mathrm{E}_z \log D(G(z)).$$

For a fixed generated sample, raising $D(G(z))$ from 0.2 to 0.8 reduces this loss from about 1.609 to 0.223. The objective explicitly encourages generated samples that the discriminator regards as real.

At the idealized distribution-matching equilibrium with balanced real/fake sampling, the optimal discriminator returns $1/2$. This means it cannot distinguish the two distributions, not that each generated image is only half correct. In actual training, $D \approx 1/2$ alone is not proof of success: an untrained or ineffective discriminator can also be uncertain.

5.5.5 GAN difficulties and image translation

Teacher slides: 5.5. Generative Models, pp. 13–16.

Mode collapse and unstable training

Mode collapse means the generator produces too little diversity: many different latent inputs map to the same or a small set of outputs that fool the current discriminator. Individual outputs may look realistic while important modes of the data distribution are absent.

Generator and discriminator objectives change as the other model changes. This can cause oscillations and imbalance. A very strong discriminator can produce weak generator gradients under some losses, especially the original saturating formulation. Alternative losses and training strategies address parts of these problems; no single discriminator score diagnoses all of them.

CycleGAN

CycleGAN illustrates unpaired translation between domains A and B. It learns mappings $G : A \to B$ and $F : B \to A$ with adversarial losses that encourage target-domain realism and cycle-consistency terms such as

$$\|F(G(x)) - x\|_1.$$

The cycle encourages translated content to support reconstruction of the original. Unpaired means a training image in A need not have an exact corresponding target image in B. Cycle consistency does not by itself uniquely determine the intended semantic mapping.

Style transfer and other image-editing tasks can use different losses and architectures; not every image stylization method is a GAN. The important connection is that a learned realism or perceptual signal can complement a direct pixelwise objective.

5.5.6 Diffusion: learn to reverse a noise process

Teacher slides: 5.5. Generative Models, pp. 17–20.

Forward noising

The forward process gradually adds Gaussian noise to a clean observation x_0 until the result is close to noise. A standard formulation writes

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1 - \bar{\alpha}_t}\, \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I),$$

where $\bar{\alpha}_t$ expresses how much original signal remains at step t. The schedule is specified rather than learned by a discriminator.

For a toy scalar with $\bar{\alpha}_t = 0.64$, $x_0 = 2$ and a chosen noise sample $\epsilon = 1$, we get

$$x_t = 0.8(2) + 0.6(1) = 2.2.$$

A noisy value can be larger than the clean value: noise is a random perturbation, not simply a deterministic shrinking of every pixel.

Training the denoiser

A common objective trains a network $\epsilon_\theta(x_t, t, c)$ to predict the sampled noise from a noisy example, its timestep and optional conditioning c:

$$L_{\text{noise}} = \mathrm{E}\left[\|\epsilon - \epsilon_\theta(x_t, t, c)\|^2\right].$$

Because the training procedure generated ϵ, it knows the target without a human labeling the noise. For true scalar noise 0.3 and prediction 0.1, squared error is 0.04.

The timestep tells the network how noisy the input is expected to be. Training commonly samples a timestep and constructs the corresponding noisy input directly; it does not require a complete sequential reverse-generation trajectory for every training example. Other parameterizations predict the clean signal or related quantities, but the central idea is learning a denoising task at different noise levels.

Sampling

Start with noise and iteratively apply a learned reverse process to obtain a structured sample. The exact update and any added sampling noise depend on the sampler. This is a learned statistical reversal, not a deterministic inversion that retrieves one original training image.

Standard many-step generation can require more network evaluations than a single GAN or VAE decoder pass. Fewer-step samplers and other improvements change that trade-off, so avoid treating one fixed step count as a universal requirement.

5.7.7 Conditioning and latent diffusion

Teacher slides: 5.5. Generative Models, pp. 5, 20–21; 5.3. Foundation models, pp. 24–25.

A conditional generator models observations given information such as a label, text prompt, image or other control. The same random seed can produce a different result under different conditioning; different seeds can produce different results for the same prompt.

In latent diffusion, an autoencoder maps images to a compressed latent space. Diffusion runs on those latents, and the decoder converts the final denoised latent back to an image:

$$\text{prompt} \to \text{text encoder} \to \text{conditioning}, \qquad \text{latent noise} \to \text{denoiser} \to \text{image decoder}.$$

The text representation can enter through attention or another conditioning mechanism. A text encoder represents language; the denoiser learns the generative transformation; the image decoder maps representations to pixels. These components have different jobs.

Working on a smaller spatial representation can reduce denoising cost. It does not eliminate the information bottleneck or guarantee that small details survive compression. The deck shows both U-Net-style and transformer-based denoising ideas in the broader course context.

5.5.8 Compare the model families

Teacher slides: 5.5. Generative Models, p. 22.

Family	Core objective	Sampling/use	Common limitation
AE	Reconstruct the input through a representation.	Encode and decode observed data.	Arbitrary random latent codes need not decode plausibly.
VAE	Probabilistic reconstruction plus latent prior regularization.	Sample from a prior and decode.	Reconstruction, latent information and sample quality trade off.
GAN	Generator competes with a real/fake discriminator.	Usually one generator pass from noise.	Adversarial instability and mode collapse.
Diffusion	Learn denoising across noise levels.	Usually iterative denoising from noise.	Sampling can require many network evaluations.

Autoencoders and VAEs generally use direct differentiable losses without an adversarial opponent. Diffusion noise regression also avoids the basic GAN minimax game. That is a useful training distinction, not a promise that any particular model is effortless to train.

Image quality is not one number: realism, diversity, reconstruction fidelity and prompt agreement are different criteria. A crisp but repetitive generator can have mode collapse; a good reconstruction model may be poor at random sampling. No unconditional ranking in the slide comparison table can replace specifying the task and evaluation.

5.5.9 Place these models in the course

Teacher slides: 5.5. Generative Models, pp. 25–26.

Artificial intelligence includes methods beyond learning, such as search and planning. Machine learning includes methods beyond neural networks, such as k-NN, SVMs and random forests. Deep learning uses multilayer neural models.

CNN, RNN and transformer describe architectural families. GAN and diffusion describe generative modeling/training frameworks that can use different neural components. NLP, computer vision and robotics are application areas, not mutually exclusive model architectures. For example, a transformer denoiser can be part of a diffusion system for computer vision.

5.5.10 Revision questions and answers

Questions

1. What is the difference between modeling $p(y \mid x)$ and generating samples of x?
2. What target trains a basic autoencoder?
3. For $x = (1, 3)$ and $\widehat{x} = (2, 1)$, what is mean squared reconstruction error?
4. Why might a randomly sampled latent decode badly in a deterministic autoencoder?
5. What does a VAE encoder output, and what does its KL term encourage?
6. For $\mu = 1, \sigma = 2, \epsilon = -0.5$, compute the reparameterized latent.
7. Which GAN network generates samples, and which evaluates real versus fake?
8. During a generator update, can gradients pass through a discriminator whose parameters are fixed?
9. What is mode collapse?
10. Does a diffusion model learn the prescribed forward noise schedule or a reverse denoising task?
11. Where does the target noise come from in noise-prediction training?
12. In latent diffusion, what do the text encoder, denoiser and image decoder each do?

Answers

1. The former predicts labels conditional on an observation; the latter models how to produce observations, possibly with conditioning.
2. The original input, compared with its reconstruction.
3. $(1 + 4)/2 = 2.5$.
4. The encoded data need not fill or match the chosen random sampling distribution.
5. A distribution's parameters, commonly mean and scale; KL encourages compatibility with a chosen prior.
6. $1 + 2(-0.5) = 0$.
7. The generator G and discriminator D, respectively.
8. Yes; gradients with respect to the generated input must pass through D to train G.
9. Insufficient generated diversity, with many latent inputs producing a narrow set of outputs.
10. It learns the reverse denoising task; the forward noising process supplies training examples.
11. The random noise sampled by the training procedure is known.
12. Text features condition generation; the denoiser removes latent noise iteratively; the decoder maps the resulting latent to pixels.

5.5.11 Source guide

`5.5. Generative Models.pdf`: modeling distinctions (pp. 3–6), AE/VAE (pp. 7–8), GAN objectives and difficulties (pp. 9–13), translation/style applications (pp. 14–16), diffusion (pp. 17–21), comparison (p. 22), and the course hierarchy (pp. 25–26). The VAE/KL, GAN-loss and diffusion equations make the slide concepts explicit; the numerical mini-examples are study applications, not additional teacher-set problems.